

SILICON DREAMS

Study Guide

MODULE 3 · WEEK 3 · CM-HW-101-M3

SG-M3-04 pairs with TB-M3-04

Clock Gating and Power Lab

Wrap `sky130_fd_sc_hd__dlclkp`, express it in SDC, and measure dynamic power drop.

Learning objectives

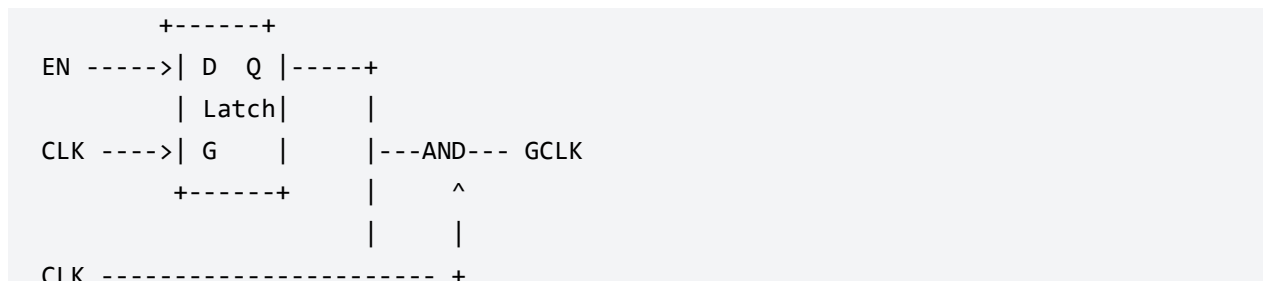
- Instantiate `sky130_fd_sc_hd__dlclkp_1` as an ICG cell and understand its latch-then-AND behaviour.
- Write ``create_generated_clock`` and ``set_clock_groups -physically_exclusive`` SDC lines correctly.
- Run OpenSTA before and after clock gating; read the power estimate delta.
- Avoid the #1 clock-gating pitfall: launching a gated clock on a datapath that also uses the ungated clock.

Prep checklist

- ☐ SG-M3-03 arbiter fairness test green.
- ☐ TB-M3-04 read. You can distinguish dynamic vs static power.
- ☐ OpenSTA accessible at ``librelane/scripts/openroad/sta.tcl`` — the run script wraps this.

Step 1 · Anatomy of the ICG cell

The SKY130 high-density library ships a dedicated integrated clock-gate cell, ``sky130_fd_sc_hd__dlclkp_1``. Its internal structure:



The level-sensitive latch captures EN while CLK is low. The AND gate then produces GCLK only when both EN is held and CLK is high. This prevents the glitch that would happen if EN were AND-ed directly with CLK.

Step 2 · Wrap it in src/clock_gate.v

```
module clock_gate (
    input  wire clk,
    input  wire enable,
    output wire gclk
);
    sky130_fd_sc_hd__dlclkp_1 u_icg (
        .CLK (clk),
        .GCLK(gclk),
        .GATE(enable)
    );
endmodule
```

Why wrap at all? Two reasons. First, the shim lets you swap in a behavioural ``assign gclk = clk & enable_latched;` for simulation on non-SKY130 environments. Second, it centralises the synthesis pragma you may need if the tool doesn't auto-infer the ICG.

Step 3 · Simulation model

```
// Behavioural alternative – used when SKY130 library is not available
module clock_gate (
    input  wire clk,
    input  wire enable,
    output wire gclk
);
    reg enable_latched;
    always @(*) if (!clk) enable_latched = enable;
    assign gclk = clk & enable_latched;
endmodule
```

Your simulation must match the ICG's latch-then-AND behaviour. The behavioural model above does exactly that.

Step 4 · SDC constraints for the gated clock

Without SDC, the tool cannot distinguish clk from gclk and will over-constrain both. Add to ``librelane/constraints.sdc`:

```
# Primary clock (external pad)
create_clock -name clk_ext -period 10.0 [get_ports clk]
```

```
# Generated clock from the ICG cell output
create_generated_clock -name gclk \
    -source [get_ports clk] \
    -divide_by 1 \
    -add \
    [get_pins u_clock_gate/u_icg/GCLK]

# Prevent false-path analysis between the two domains
set_clock_groups -physically_exclusive \
    -group {clk_ext} \
    -group {gclk}
```

E8873C

CRITICAL: The physically_exclusive group is what prevents the STA engine from chasing a setup path that can never occur (because gclk is gclk only when enable is high, and the receiving flop sees gclk). Miss this line and your WNS report will lie to you.

Step 5 · Run OpenSTA before and after

```
# Before applying the ICG
./scripts/run-librelane.sh --tag pre-icg

# After applying the ICG + SDC
./scripts/run-librelane.sh --tag post-icg

# verifying clock gating
./scripts/verify_clock_gating.sh

# POWER REPORT
./scripts/get_power_report.sh

# Diff the power estimates
diff runs/pre-icg/reports/signoff/power.rpt \
    runs/post-icg/reports/signoff/power.rpt
```

Expected: dynamic power on the internal flop cloud drops by ~20-35% when the elevator is in IDLE (which is most of the time). Static power is unchanged — clock gating has no effect on leakage.

Step 6 · The #1 pitfall

A gated clock **MUST NOT** drive a flop whose D-side data crosses into the ungated domain without a synchroniser. This is the same rule as asynchronous clock domains.

```
// WRONG — cross-domain without a synchroniser
```

```

always @(posedge clk)      data_main  <= gated_sample;
always @(posedge gclk)     gated_sample <= new_sample;

// CORRECT – synchronise at the crossing
always @(posedge clk) begin
    sync_ff_1 <= gated_sample;
    data_main <= sync_ff_1;
end

```

Troubleshooting

WNS gets WORSE after adding ICG	You forgot the <code>physically_exclusive</code> group. STA is analysing a nonsense path.
Simulation mismatches post-synthesis	Your behavioural model is not latch-then-AND. Review Step 3.
Tool inserts an extra ICG in CTS	It is auto-cloning your gating enable. Force only one ICG with a synthesis attribute or by setting <code>`CTS_CLK_MAX_WIRE_LENGTH`</code> .
Power report doesn't show a drop	Your enable is always 1. Check that the IDLE-state detector actually pulls enable low — write a cocotb waveform assertion.

Reflection

- Record the WNS and total dynamic power before and after your ICG change. You'll need both in your escape log.
- Write a one-paragraph explanation of why ``set_clock_groups -physically_exclusive`` (not -asynchronous) is the correct flag for gated clocks.